

Universidad Autónoma de Santo Domingo

PRIMADA DE AMERICA / Fundada el 28 de octubre de 1538

Nombre:

Wilmy.

Apellidos:

Medina Batista.

Matrícula:

100681393.

Asignatura:

Laboratorio De Lenguaje De Programación III.

Tema:

JSON y Sus Usos.

Sección:

01.

Profesor:

Radhames Silverio González.

Fecha:

30/10/2026.

ÍNDICE

INTRODUCCIÓN 3
DESARROLLO: 4
1. ¿Qué es JSON? 4
2. ¿Cuál es el propósito de JSON? 4
Entre sus propósitos más importantes se encuentran: 4
3. Características principales de JSON. 5
4. Ventajas y desventajas del uso de JSON. 6
Ventajas: 6
Desventajas: 6
5. Diferencia entre JSON y XML. 7
6. Usos actuales de JSON. 7
7. Ejemplo sencillo de un objeto JSON y explicación de sus partes. 8
Explicación de las partes principales: 9
CONCLUSIÓN 10
REFERENCIAS BIBLIOGRÁFICAS 11

INTRODUCCIÓN

En el desarrollo de aplicaciones informáticas, los datos necesitan organizarse de una manera que permita almacenarlos, consultarlos y compartirlos entre programas diferentes. Para lograrlo, se utilizan formatos estructurados que establecen reglas sobre cómo representar la información. Uno de los formatos más utilizados para esta tarea es JSON, sigla de JavaScript Object Notation, una notación de texto que permite representar datos de manera clara y compacta.

JSON se caracteriza por utilizar una estructura sencilla basada en objetos, arreglos y pares de clave y valor. Aunque su nombre procede de JavaScript, no depende de ese lenguaje, por lo que puede utilizarse desde aplicaciones desarrolladas con distintas tecnologías. Su sintaxis está definida por estándares que permiten que los sistemas generen y procesen información de forma interoperable (Bray, 2017; Ecma International, 2017).

El propósito de esta investigación es explicar qué es JSON, para qué sirve, cuáles son sus características, ventajas y limitaciones, y en qué se diferencia de XML. También se describen sus usos actuales en las API, las aplicaciones web, las aplicaciones móviles y los servicios web. Finalmente, se presenta un ejemplo sencillo de un objeto JSON para identificar sus componentes principales y comprender cómo se organiza la información.

DESARROLLO:

1. ¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato de intercambio de datos basado en texto que permite representar información estructurada mediante una sintaxis sencilla. Fue derivado de la notación de objetos de JavaScript, pero puede utilizarse con muchos otros lenguajes de programación. Su objetivo es proporcionar una forma común de escribir datos para que una aplicación pueda enviarlos y otra pueda interpretarlos sin depender de una tecnología específica (Bray, 2017).

Un documento JSON puede representar información mediante objetos, arreglos, cadenas de texto, números, valores booleanos y el valor nulo. Un objeto se escribe entre llaves y contiene pares de nombres y valores; un arreglo se escribe entre corchetes y reúne una secuencia ordenada de valores. Esta organización permite representar tanto datos sencillos como estructuras más complejas formadas por objetos y arreglos anidados.

JSON no es un lenguaje de programación ni una base de datos. Es una notación para representar y transportar datos. Para utilizar la información, una aplicación normalmente la convierte desde el texto JSON a las estructuras de datos que maneja su lenguaje y, cuando necesita enviarla, puede volver a serializarla en JSON. Ecma International (2017) define las reglas sintácticas del formato, mientras que el significado de los datos depende del acuerdo entre las aplicaciones que los intercambian.

2. ¿Cuál es el propósito de JSON?

El propósito principal de JSON es facilitar el intercambio de información estructurada entre sistemas informáticos. Por ejemplo, una aplicación móvil puede solicitar los datos de un usuario a un servidor y recibir una respuesta JSON que incluya su nombre, correo electrónico y preferencias. La aplicación interpreta esos datos y los presenta en la pantalla sin que el usuario tenga que conocer el formato utilizado para transportarlos.

Entre sus propósitos más importantes se encuentran:

- **Intercambiar información entre aplicaciones:** Permite enviar y recibir datos entre un cliente y un servidor o entre servicios que utilizan tecnologías distintas.
- **Organizar los datos:** Las claves identifican el significado de los valores, mientras que los objetos y arreglos permiten agrupar información relacionada.
- **Facilitar el procesamiento automático:** Las bibliotecas disponibles en numerosos lenguajes permiten analizar y generar JSON con procedimientos conocidos.
- **Reducir la complejidad de los mensajes:** Su sintaxis evita muchas etiquetas de apertura y cierre, por lo que suele producir mensajes compactos.

- **Apoyar la comunicación mediante API:** Muchas interfaces de programación utilizan JSON para representar solicitudes, respuestas y resultados.

Por estas razones, JSON resulta conveniente cuando dos sistemas necesitan intercambiar datos de forma clara y práctica. Sin embargo, el formato solo establece cómo se escribe la información; los sistemas también deben acordar qué significa cada campo y qué valores son aceptables (Bray, 2017).

3. Características principales de JSON.

Sintaxis sencilla: JSON utiliza un conjunto reducido de símbolos, como llaves, corchetes, dos puntos y comas. Esta sintaxis permite escribir estructuras de datos que suelen ser fáciles de leer y revisar.

Pares clave-valor: Los objetos JSON agrupan valores identificados por nombres escritos como cadenas de texto. Por ejemplo, la clave "nombre" puede asociarse con el valor "Laura", de modo que la aplicación reconoce qué información representa ese valor.

Tipos de datos definidos: JSON permite representar cadenas de texto, números, valores booleanos (true y false), el valor null, objetos y arreglos. No incorpora directamente tipos como fechas, funciones o comentarios; cuando se necesita transmitir una fecha, por ejemplo, suele acordarse una representación textual.

Objetos y arreglos anidados: Un objeto puede contener otro objeto o un arreglo, y un arreglo puede reunir diferentes valores. Esta característica permite describir registros, listas y estructuras con varios niveles de información.

Independencia del lenguaje: Aunque JSON surgió a partir de JavaScript, su sintaxis no está limitada a ese lenguaje. Puede generarse y procesarse en entornos como Java, Python, C#, PHP y muchos otros.

Formato de texto e interoperabilidad: Los datos se transmiten como texto y suelen codificarse en UTF-8 cuando circulan por una red. Esto facilita su intercambio entre sistemas, siempre que todos respeten las reglas del formato y comprendan el significado de los campos (Bray, 2017).

Sintaxis estricta: Los nombres de las propiedades y las cadenas deben escribirse entre comillas dobles; los pares se separan con comas y se utiliza dos puntos entre una clave y su valor. Los comentarios y las comas finales no forman parte de la sintaxis estándar, por lo que pueden causar errores al analizar un documento.

Validación complementaria: JSON permite definir la estructura básica de los datos, pero la sintaxis por sí sola no determina si un campo es obligatorio, si un número debe estar dentro de cierto rango o si un correo electrónico tiene un formato adecuado. Para aplicar esas reglas, los sistemas pueden utilizar validaciones de la aplicación o tecnologías adicionales, como JSON Schema.

4. Ventajas y desventajas del uso de JSON.

Ventajas:

- **Legibilidad:** Su estructura es relativamente fácil de leer, por lo que una persona puede revisar los datos en un editor de texto sin necesitar una herramienta especializada.
- **Tamaño generalmente compacto:** En muchos casos requiere menos texto que XML para representar los mismos datos, lo que puede reducir el volumen de los mensajes transmitidos.
- **Facilidad de integración:** Existe soporte para JSON en numerosos lenguajes, plataformas y bibliotecas, lo que facilita su incorporación a diferentes proyectos.
- **Adecuado para aplicaciones web:** JavaScript puede trabajar directamente con estructuras equivalentes a JSON, y los navegadores y servidores cuentan con herramientas ampliamente disponibles para procesarlo.
- **Estructuras flexibles:** Los objetos y arreglos pueden combinarse para representar datos relacionados, listas y registros con diferentes niveles.
- **Amplio uso en API:** Su sintaxis facilita crear respuestas y solicitudes que pueden ser interpretadas por aplicaciones web, móviles y otros servicios.

Desventajas:

- **Tipos limitados:** El formato no define de manera nativa fechas, horas, datos binarios, funciones ni tipos especializados. Es necesario acordar cómo representar esos valores.
- **Validación no incorporada de forma completa:** La sintaxis permite saber si el texto es JSON válido, pero no garantiza por sí sola que los datos cumplan las reglas de negocio de una aplicación.
- **Posible ambigüedad en los acuerdos:** Dos sistemas pueden interpretar de manera diferente un campo si no existe una documentación clara sobre su significado, formato o unidades.
- **No siempre es el más adecuado para documentos complejos:** Cuando se requieren mecanismos establecidos de validación, espacios de nombres o transformaciones documentales, XML puede ofrecer herramientas más apropiadas.
- **Riesgos de seguridad por un manejo incorrecto:** Una aplicación debe validar los datos recibidos y evitar procesarlos mediante mecanismos que ejecuten código. JSON debe tratarse como datos, no como instrucciones.

En consecuencia, JSON no es superior a todos los demás formatos en cualquier situación. Suele ser una opción práctica para intercambiar datos en aplicaciones y API, pero la elección debe considerar la estructura requerida, las reglas de validación, la compatibilidad y las necesidades de seguridad del proyecto.

5. Diferencia entre JSON y XML.

JSON y XML son formatos de texto utilizados para representar e intercambiar información, pero organizan los datos de manera diferente. JSON emplea objetos, arreglos y pares clave-valor; XML utiliza etiquetas de apertura y cierre, elementos, atributos y una estructura jerárquica. Ambos pueden representar datos anidados y ser procesados por aplicaciones, pero sus características hacen que resulten más convenientes en contextos distintos.

Aspecto	JSON	XML
Estructura	Objetos, arreglos y pares clave-valor.	Elementos, etiquetas, atributos y jerarquías.
Sintaxis	Generalmente compacta y con menos caracteres.	Suele requerir etiquetas de apertura y cierre.
Tipos de datos	Incluye cadenas, números, booleanos, null, objetos y arreglos.	El contenido se representa mediante texto; los tipos pueden definirse mediante esquemas.
Validación	Puede complementarse con reglas de la aplicación o JSON Schema.	Puede utilizar DTD o XML Schema para definir y validar estructuras.
Uso frecuente	API web, aplicaciones web y móviles.	Documentos estructurados, ciertos servicios empresariales y sistemas existentes.
Legibilidad	Suele ser breve para representar objetos y listas.	Las etiquetas hacen explícita la estructura, aunque aumentan la longitud.

La diferencia también se aprecia en la manera de escribir un mismo dato. En JSON, un nombre y un apellido pueden representarse como propiedades de un objeto. En XML, los mismos valores se expresan mediante elementos con etiquetas. Ninguno de los dos formatos es siempre mejor: JSON suele ser cómodo para mensajes de API y estructuras de objetos, mientras que XML mantiene utilidad cuando los sistemas requieren tecnologías documentales o de validación específicas (Bray, 2017; World Wide Web Consortium [W3C], 2008).

6. Usos actuales de JSON.

API y comunicación entre aplicaciones:

Muchas API utilizan JSON para enviar datos entre un cliente y un servidor. Por ejemplo, una aplicación puede solicitar una lista de productos y recibir un arreglo de objetos con identificadores, nombres y precios. El cliente interpreta la respuesta y utiliza la información para actualizar la pantalla. La cabecera de tipo de contenido application/json permite indicar que el cuerpo de un mensaje contiene datos JSON.

Aplicaciones web:

En las aplicaciones web, JSON se utiliza para comunicar la interfaz que se ejecuta en el navegador con los servicios del servidor. También puede emplearse para guardar configuraciones, representar catálogos, filtrar resultados o cargar datos sin volver a descargar toda la página. Su compatibilidad con las herramientas de JavaScript facilita convertir el texto recibido en estructuras que la aplicación pueda utilizar.

Aplicaciones móviles:

Las aplicaciones móviles recurren a JSON cuando necesitan consultar cuentas, productos, noticias, mapas, mensajes o información almacenada en servidores. La aplicación envía una solicitud y recibe una respuesta que contiene los datos necesarios. Debido a que JSON no está ligado a un sistema operativo o lenguaje específico, un mismo servicio puede atender clientes desarrollados para diferentes plataformas.

Servicios web e integración de sistemas:

JSON se emplea en servicios web que intercambian información mediante solicitudes y respuestas. También puede servir para conectar componentes de un sistema, automatizar procesos y comunicar aplicaciones que se ejecutan en servidores diferentes. En estos casos, es importante documentar las propiedades, los tipos de valores, los campos obligatorios y la forma en que se comunican los errores.

Archivos de configuración y almacenamiento de datos:

Algunos programas guardan en archivos JSON preferencias, opciones de ejecución, parámetros y datos estructurados. También se utiliza para importar o exportar información entre herramientas. Aunque JSON puede servir para representar datos que luego se almacenan, no sustituye por sí solo a un sistema gestor de bases de datos: cada tecnología cumple una función distinta.

Estos usos explican por qué JSON es frecuente en el desarrollo de software moderno. Su sencillez facilita la comunicación entre sistemas, pero para que la integración sea confiable se necesita una estructura documentada, validación de datos y un tratamiento seguro de la información recibida (Ecma International, 2017; Bray, 2017).

7. Ejemplo sencillo de un objeto JSON y explicación de sus partes.

A continuación se presenta un ejemplo de un objeto JSON que almacena datos básicos de un estudiante. La información es ficticia y se utiliza únicamente con fines educativos:

```
{
  "matricula": "2026-001",
  "nombre": "Laura",
  "apellido": "Martínez",
  "carrera": "Informática",
```

```
  "asignatura": "Laboratorio de Lenguaje de Programación
III",
  "activo": true,
  "calificaciones": [90, 95, 88],
  "contacto": {
    "correo": "laura@example.com",
    "teléfono": null
  }
}
```

Explicación de las partes principales:

- **Llaves { }:** Delimitan el objeto completo. Todas las propiedades del estudiante se encuentran dentro de estas llaves.
- **Claves:** Nombres como "matricula", "nombre", "carrera" y "activo" identifican la información que se almacena. En JSON estándar, las claves deben escribirse entre comillas dobles.
- **Valores de texto:** "2026-001", "Laura", "Martínez" y los demás textos son cadenas de caracteres. Se escriben entre comillas dobles.
- **Valor booleano:** True indica que el estudiante está activo. Los valores booleanos se escriben en minúscula y no llevan comillas.
- **Arreglo [ ]:** La propiedad "calificaciones" contiene una lista ordenada de números. Los elementos del arreglo se separan con comas.
- **Objeto anidado:** La propiedad "contacto" contiene otro objeto con información de correo electrónico y teléfono. Esto permite agrupar datos relacionados.
- **Valor null:** En "teléfono", null indica que actualmente no se proporciona un valor. No debe escribirse entre comillas porque dejaría de ser el valor nulo y pasaría a ser texto.
- **Comas y dos puntos:** Los dos puntos separan cada clave de su valor, y las comas separan las propiedades o los elementos de un arreglo. No se coloca una coma después de la última propiedad.

Para que el ejemplo sea válido, las cadenas y las claves deben usar comillas dobles, los valores deben corresponder a la sintaxis de JSON y las comas deben ubicarse correctamente. A diferencia de algunos objetos escritos directamente en JavaScript, JSON estándar no admite comentarios ni funciones. Estas reglas permiten que los analizadores compatibles interpreten la estructura de manera consistente (Bray, 2017).

En una aplicación real, el programa podría leer este texto JSON y convertirlo en una estructura de datos para mostrar el nombre del estudiante, consultar su estado o recorrer

sus calificaciones. También podría enviar el objeto a un servicio, siempre que el sistema receptor conozca el significado de cada propiedad y valide los datos según sus necesidades.

CONCLUSIÓN

JSON es un formato de intercambio de datos que permite representar información mediante una estructura sencilla basada en objetos, arreglos y pares clave-valor. Su independencia respecto de un lenguaje de programación y su sintaxis compacta facilitan que aplicaciones diferentes puedan intercambiar información de manera organizada.

A lo largo de la investigación se identificó que JSON ofrece ventajas como la legibilidad, la facilidad de procesamiento, la compatibilidad con numerosas tecnologías y su utilidad en las API. También presenta limitaciones: sus tipos de datos son reducidos, la sintaxis no valida por sí sola las reglas de negocio y es necesario manejar con cuidado los datos que llegan desde otros sistemas. Por ello, su uso debe acompañarse de documentación, validaciones y medidas de seguridad.

Asimismo, se observó que JSON y XML cumplen funciones semejantes en el intercambio de información, aunque utilizan estructuras diferentes. JSON suele ser práctico para las comunicaciones entre aplicaciones web y móviles, mientras que XML conserva utilidad en documentos estructurados y entornos que dependen de sus herramientas específicas. La elección entre ambos debe responder a los requisitos del proyecto y a la compatibilidad de los sistemas involucrados.

En conclusión, conocer JSON es importante para quienes estudian programación porque permite comprender cómo se representan y transportan datos en muchos sistemas actuales. El ejemplo presentado demuestra que, mediante claves, valores, arreglos y objetos anidados, se puede describir información de forma clara para que diferentes aplicaciones puedan procesarla.

REFERENCIAS BIBLIOGRÁFICAS

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) Data Interchange Format (RFC 8259).* Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8259.html

Ecma International. (2017). *The JSON data interchange syntax (ECMA-404, 2nd ed.).* https://ecma-international.org/publications-and-standards/standards/ecma-404/

World Wide Web Consortium. (2008, 26 de noviembre). *Extensible Markup Language (XML) 1.0 (Fifth Edition).* https://www.w3.org/TR/xml/